

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Ingeniería

Carrera de Ingeniería en Software

SGED

Sistema de Gestión para la
Escuela Deportiva ProFútbol

Práctica Experimental — Unidad IV

Informe Técnico Final

Proyecto Fin de Curso — Aplicaciones Web

Asignatura: Aplicaciones Web (Quinto nivel)

Docente: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

Periodo: PPA 2026-2027

Etiqueta: v1.0.0-rc (candidata a versión estable)

Commit del corte: 23deb19

Repositorio: https://github.com/Richiflo0o/SGED_APPWEB_PE_U4.git

Integrantes (grupo de la práctica):

Velez Lopez Ricardo Elias

[Pendiente de Integrante A]

[Pendiente de Integrante B]

Quevedo — Los Ríos — Ecuador — 2026

Índice

Resumen	3
Abstract	3
1 Introducción	4
2 Metodología	4
3 La API REST documentada (Bloques 1–2)	4
3.1 Anotaciones OpenAPI (Bloque 2)	5
4 Consumo de API externa con caché Redis (Bloque 5.1)	5
4.1 Endpoint y contrato	5
4.2 Estrategia de caché (<i>cache-aside</i> , TTL 24h)	5
4.3 Manejo de errores ante fallos de la API externa	6
4.4 Evidencia de una petición real	6
5 Consumo de Servicios Web Externos y SOAP (Bloque 5.3)	7
5.1 SOAP vs. REST: análisis técnico comparativo	7
5.2 Clientes HTTP del lado del servidor	7
5.3 Caché de respuestas externas	9
5.4 Tendencias: Jamstack y APIs en 2025	9
6 Seguridad: auditoría OWASP (Bloque 3)	10
6.1 A01 — Broken Access Control	10
6.2 A02 — Cryptographic Failures	10
6.3 A03 — Injection	10
6.4 A04 — Insecure Design	11
6.5 A05 — Security Misconfiguration	11
6.6 A06 — Vulnerable and Outdated Components	11
6.7 A07 — Identification and Authentication Failures	11
6.8 A09 — Security Logging and Monitoring Failures	11
6.9 XSS — Cross-Site Scripting	11
6.10 Resultado de la auditoría	12
7 Migraciones Flyway y reproducibilidad (Bloque 4)	12
8 Pruebas de carga (Bloque 5.3)	12
9 Reflexión y modelo de amenazas (Bloque 5.4)	13
9.1 Modelo de amenazas y decisiones de diseño	13
9.2 Lección transversal: una medición que no se ejecuta contra el sistema real no es evidencia	13
9.3 Contribución de la práctica	13
10 Autoría y honestidad académica	13
11 Conclusiones	14
12 Trabajo futuro	14
13 Declaración de uso de inteligencia artificial	14

A Anexo: Taxonomía de Bloom aplicada a la práctica**15**

Resumen

Este informe documenta la Práctica Experimental de la Unidad IV del Proyecto Fin de Curso (PFC): la verificación técnica del sistema **SGED** (Sistema de Gestión para la Escuela Deportiva ProFútbol), aplicación web MVC construida con Spring Boot 3.2.5, Angular y PostgreSQL 16. El trabajo se organizó en bloques con autoría verificable por *commit*: exposición de la API REST documentada con Swagger/OpenAPI en `/api/docs` (Bloque 1), anotaciones `@Tag/@Operation/@Schema` en controladores y DTOs (Bloque 2), auditoría de seguridad alineada con el OWASP Top 10 ampliada a A04, A06, A07, A09 y XSS (Bloque 3), migraciones Flyway V1–V7 alineadas con el esquema y el contenedor Docker reproducible (Bloque 4), consumo de una API externa de datos de fútbol con caché Redis (Bloque 5.1) y la medición de rendimiento con k6 (Bloque 5.3). La seguridad se evidencia con peticiones `curl` reproducibles guardadas en `docs/mediciones/sec/`, incluidas las cabeceras de seguridad, el bloqueo por fuerza bruta (429 tras seis intentos fallidos) y la protección ante XSS con CSP y tipo de contenido `application/json`. Este informe declara además, de forma explícita y transparente, el uso de herramientas de asistencia por inteligencia artificial en el desarrollo.

Abstract

This report documents the Experimental Practice of Unit IV of the Final Course Project (PFC): the technical verification of the **SGED** system (Sports Management System for the ProFútbol Sports School), a MVC web application built with Spring Boot 3.2.5, Angular and PostgreSQL 16. The work was organized in blocks with authorship verifiable by *commit*: REST API exposure documented with Swagger/OpenAPI at `/api/docs` (Block 1), `@Tag/@Operation/@Schema` annotations in controllers and DTOs (Block 2), a security audit aligned with the OWASP Top 10 extended to A04, A06, A07, A09 and XSS (Block 3), Flyway migrations V1–V7 aligned with the schema and a reproducible Docker container (Block 4), external football-data.org API consumption with Redis caching (Block 5.1), and load testing with k6 (Block 5.3). Security is evidenced with reproducible `curl` requests stored in `docs/mediciones/sec/`, including security headers, brute-force blocking (429 after six failed attempts) and XSS protection through CSP and `application/json` content type. This report additionally declares, explicitly and transparently, the use of artificial intelligence assistance tools during development.

1 Introducción

La Práctica Experimental de la Unidad IV exige aplicar sobre el Proyecto Fin de Curso las competencias de la asignatura Aplicaciones Web: una aplicación web MVC completa con los módulos de la Entrega 1B funcionando, API REST documentada con Swagger, consumo de una API externa con caché Redis, seguridad según el OWASP Top 10, pruebas de carga y el informe técnico final. Este repositorio (https://github.com/Richiflo0o/SGED_APPWEB_PE_U4.git) es un clon derivado de DarwinSM21/SGED_APPWEB que conserva el historial completo de las entregas 1A–3 y registra el trabajo de la práctica en la rama `feature/u4-docs-seguridad`.

El trabajo se dividió en bloques. Los bloques 1–4 y la auditoría de seguridad fueron desarrollados y verificados por Ricardo Velez Lopez (autoría verificable en el historial de `git`); los bloques 5.1 (consumo de la API externa) y 5.3 (pruebas de carga) son responsabilidad de los integrantes A y B respectivamente y quedan marcados en este informe con los espacios reservados para sus resultados. La distribución de autoría se resume en la sección 10.

2 Metodología

La práctica se desarrolló con verificación continua y evidencia reproducible: cada bloque produce artefactos versionados en el repositorio (evidencia cruda de `curl`, reportes de cobertura, salidas de `k6`), de modo que el profesor puede reproducir cada resultado con los comandos del *Makefile* sin depender de la palabra de los integrantes.

Comando	Acción
<code>make up</code>	Levanta el sistema completo (Docker Compose)
<code>make test</code>	Pruebas JUnit 5 + cobertura JaCoCo
<code>make bench</code>	3 corridas k6 (50 VUs, 30s) + análisis IC 95 %
<code>make audit</code>	Auditoría OWASP (A01–A07, A09, XSS) + SQL dinámico
<code>make clean</code>	Limpia contenedores, volúmenes y compilaciones

Las credenciales semilla para probar el sistema son `admin / Admin2026!` (definidas en `db/seed.sql`). El certificado TLS de `https://localhost:8443` es autofirmado, generado solo para desarrollo y evaluación.

3 La API REST documentada (Bloques 1–2)

SGED expone su API REST bajo el prefijo `/api`. La documentación se publica con `springdoc-openapi 2.3.0` en dos rutas:

- **Swagger UI:** GET `http://localhost:8080/api/docs` — interfaz interactiva que permite explorar y probar cada *endpoint*.
- **OpenAPI 3.0 en JSON:** GET `http://localhost:8080/api/api-docs` — documento consumible por herramientas y generadores de clientes.

Ambas rutas están permitidas en `SecurityConfig.java` y accesibles sin autenticación para su consulta. La configuración vive en `application.yml`:

```

1 springdoc:
2   api-docs:
3     path: /api/api-docs
4   swagger-ui:
5     path: /api/docs

```

Listing 1: Rutas de la documentación (Bloque 1)

3.1 Anotaciones OpenAPI (Bloque 2)

Cada controlador del dominio académico, deportivo y seguridad está anotado con `@Tag` (agrupación), y cada *endpoint* con `@Operation` (descripción, resumen y respuestas posibles). Los DTOs de entrada y salida (`EstudianteRequest`, `EstudianteResponse`, `LoginRequest`, `RegisterRequest`, `SesionResponse`, etc.) usan `@Schema` para documentar campos, tipos y restricciones. Ejemplo:

```

1 @Tag(name = "Estudiantes",
2     description = "CRUD de estudiantes con paginacion, soft delete y conteos
3     por categoria")
4 public class EstudianteController {
5
6     @GetMapping
7     @PreAuthorize("hasAnyRole('ADMINISTRADOR', 'ENTRENADOR', 'USER')")
8     @Operation(summary = "Listar estudiantes paginados",
9         description = "Devuelve una pagina de estudiantes ordenables.")
10    @ApiResponse({
11        @ApiResponse(responseCode = "200", description = "Pagina de estudiantes"
12        ),
13        @ApiResponse(responseCode = "403", description = "Rol sin permiso")
14    })
15    public ResponseEntity<EstudiantePageResponse<EstudianteResponse>> listar(
16        @RequestParam(defaultValue = "0") int page,
17        @RequestParam(defaultValue = "10") int size,
18        @RequestParam(defaultValue = "idEstudiante,asc") String[] sort) {
19        ...
20    }
21 }

```

Listing 2: Anotaciones OpenAPI en EstudianteController

La colección de Postman (`docs/postman/coleccion.json`) documenta el uso manual de la API, y el atributo de calidad de la documentación se mapea en `docs/iso25010-atributos-calidad.md` sobre el modelo ISO/IEC 25010 [8].

4 Consumo de API externa con caché Redis (Bloque 5.1)

Esta sección documenta la integración de la API externa de datos de fútbol `football-data.org` [6] consumida por SGED, con la estrategia de caché en Redis para limitar la cantidad de llamadas y el tiempo de respuesta. El análisis comparativo de SOAP frente a REST, de clientes HTTP y de la justificación general del patrón de caché se desarrolla en la sección 5; esta sección se centra en el contrato del *endpoint*, la implementación real y la evidencia de una petición contra el sistema levantado.

4.1 Endpoint y contrato

GET `/api/futbol/posiciones?liga={codigo}` resuelve la tabla de posiciones de una liga de fútbol. Es público (`SecurityConfig.permitAll` sobre `/api/futbol/**`): son datos deportivos de terceros sin información personal, el mismo criterio que el proyecto ya aplica a `/api/docs` y `/actuador/health`. La respuesta sigue el sobre `ApiResponse<TablaPosicionesDto>` `{success, data, message, errors, meta}`, con `meta.origen` $\in \{\text{api, cache, fallback}\}$ para poder distinguir de dónde vino cada respuesta.

4.2 Estrategia de caché (*cache-aside*, TTL 24 h)

La caché es manual, con `StringRedisTemplate` y un `ObjectMapper` (no `@Cacheable`, para que el patrón sea explícito y demostrable), sobre la clave `api_externa:standings:{LIGA}` con TTL de 86400s (24 h, justificado en la sección 5 frente a otros dominios de datos):

```

1 private TablaPosicionesDto leerCache(String clave, Map<String, Object> meta) {
2     try {
3         String json = redisTemplate.opsForValue().get(clave);
4         if (json == null) return null;
5         return objectMapper.readValue(json, TablaPosicionesDto.class);
6     } catch (DataAccessException e) {
7         meta.put("cache", "no disponible"); // Redis caído: sigue contra la
8         API
9         return null;
10    } catch (JsonProcessingException e) {
11        return null; // entrada corrupta: se ignora
12    }
13 }
14 private void guardarCache(String clave, TablaPosicionesDto tabla, Map<String,
15     Object> meta) {
16     try {
17         String json = objectMapper.writeValueAsString(tabla);
18         redisTemplate.opsForValue().set(clave, json, Duration.ofSeconds(props.
19             ttlCacheSeconds()));
20     } catch (DataAccessException e) {
21         meta.put("cache", "no disponible");
22     } catch (JsonProcessingException e) { /* no se pudo serializar */ }
23 }

```

Listing 3: Lectura y escritura de la cache (FutbolService.java)

4.3 Manejo de errores ante fallos de la API externa

La política de reintentos y la clasificación por tipo de fallo se detallan en la sección 5. En síntesis: cualquier fallo del proveedor (*timeout*, 4xx, 5xx) hace que `FutbolService` degrade a datos de referencia con HTTP 200, `meta.origen = "fallback"` y `meta.degradado = true`, en vez de propagar el error al cliente. El Motivo interno (TIMEOUT, RATE_LIMIT, NO_AUTORIZADO, NO_ENCONTRADO, SERVIDOR) se registra únicamente en el log (WARN), nunca se filtra al cliente. Un código de liga fuera del catálogo del plan gratuito se rechaza con 400 antes de tocar la red, para no gastar cuota en un parámetro inválido del cliente.

4.4 Evidencia de una petición real

La evidencia se generó ejecutando la carpeta *05 API externa* de `docs/postman/coleccion.json` con `newman` (Postman CLI) contra el sistema real levantado con `docker compose` (Postgres, Redis y el backend), con una `FOOTBALL_API_KEY` real configurada en `.env`. Evidencia cruda completa en `docs/mediciones/api-externa/evidencia-postman.txt`.

1. **Cache miss** (clave de Redis recién borrada, primera petición) — GET `/api/futbol/posiciones?liga=PL` 200 OK en **653 ms** (llamada real de red a `football-data.org`), datos reales de la Premier League 2026/2027 (*Arsenal FC*, 85 pts en la primera posición), `meta.origen = "api"`.

2. **Cache hit** (misma petición, sin borrar la clave) — 200 OK en **54 ms**, ~12× más rápido, sin tocar la red, `meta.origen = "cache"`.

3. **TTL confirmado directo en Redis:**

```

1 $ docker exec sged_redis redis-cli TTL "api_externa:standings:PL"
2 86380

```

Listing 4: Confirmación del TTL en Redis (evidencia real)

(86400s = 24h configuradas en `football.api.ttl-cache-seconds`; 86380s se leyó a los pocos segundos de escribirse la clave, confirmando la ventana completa).

4. **Liga no soportada** (validación antes de tocar la red, esperado 400):

```

1 $ curl -s -w "\nHTTP_STATUS:%{http_code}\n" \
2   "http://localhost:8090/api/futbol/posiciones?liga=XXXXX"
3 {"type":"https://sged.uteq.edu.ec/errores/ReglaDeNegocio","title":"Bad Request",
4  "status":400,"detail":"Codigo de liga no soportado: 'XXXXX'. ...",
5  "instance":"/api/futbol/posiciones", ...}
6 HTTP_STATUS: 400

```

Listing 5: Validacion de liga invalida (evidencia real)

5 Consumo de Servicios Web Externos y SOAP (Bloque 5.3)

Esta sección analiza el consumo de servicios web externos desde una perspectiva más amplia que la implementación concreta de la sección 4: compara el paradigma SOAP frente a REST, contrasta los clientes HTTP disponibles en distintos ecosistemas de servidor, justifica la estrategia de caché adoptada y sitúa la decisión de diseño de SGED dentro de las tendencias de consumo de APIs de 2025.

5.1 SOAP vs. REST: análisis técnico comparativo

SOAP (*Simple Object Access Protocol*) es un protocolo basado en XML: cada mensaje se envuelve en un *envelope* con secciones **Header** y **Body**, la interfaz del servicio se describe formalmente con WSDL (*Web Services Description Language*), y el transporte típico es HTTP, aunque el estándar también admite SMTP o TCP crudo; cada operación se identifica con una cabecera **SOAPAction**. REST (*Representational State Transfer*), en cambio, no es un protocolo sino un estilo arquitectónico [5]: no impone un formato de mensaje (JSON es hoy el de facto), la descripción de la interfaz es opcional (OpenAPI 3.0, ya usada por SGED en la sección 4), y el transporte es siempre HTTP, aprovechando sus verbos y códigos de estado como parte del contrato [2].

Criterio	SOAP	REST
Formato del mensaje	XML obligatorio	Cualquiera (JSON de facto)
Overhead de serialización	Alto (<i>envelope</i> + esquema XSD)	Bajo (JSON compacto)
Tipado	Fuerte, vía WSDL/XSD	Débil, por convención (OpenAPI opcional)
Interoperabilidad	Alta entre plataformas empresariales	Alta en web/móvil, JSON universal
Facilidad desde JavaScript	Baja (requiere parseo XML + WS-*)	Alta (<code>fetch</code> /JSON.parse nativos)

SOAP no ha desaparecido: sigue vigente en integraciones bancarias y en organismos públicos que exponen contratos WSDL formales por razones de auditoría y compatibilidad con sistemas legados, como el propio Servicio de Rentas Internas (SRI) de Ecuador, cuyo esquema de comprobantes electrónicos (`RecepcionComprobantes/AutorizacionComprobantes`) se consume vía WSDL/SOAP. La tendencia documentada en la literatura reciente, sin embargo, es la migración progresiva de esos servicios legados hacia REST cuando la integración lo permite [4], precisamente porque el bajo *overhead* y la interoperabilidad con clientes web/móviles pesan más que la rigidez de tipado que ofrecía SOAP. SGED, al no tener restricciones de integración legada, eligió REST para consumir football-data.org [6]: la API externa ya expone JSON sobre HTTP, sin necesidad de generar un cliente a partir de un WSDL.

5.2 Clientes HTTP del lado del servidor

Todo backend que consume una API externa resuelve los mismos problemas —*timeout*, reintentos y concurrencia— con herramientas distintas según el ecosistema:

Cliente	Lenguaje	Modelo	Reintentos / backoff
GuzzleHTTP	PHP	Síncrono, con promesas opcionales	Sin <i>retry</i> nativo; requiere <code>guzzle-retry-middleware</code>
HttpClient	.NET	Síncrono/asíncrono (<code>async/await</code>)	<code>Microsoft.Extensions.Http.Polly</code> (Polly)
RestTemplate	Java (Spring)	Síncrono, bloqueante	Sin <i>retry</i> nativo; requiere <code>spring-retry</code> o bucle manual
WebClient	Java (Spring, reactivo)	No bloqueante (Reactor)	<code>.retryWhen()</code> con <code>Retry.backoff()</code> nativo

SGED usa `RestClient` (Spring 6.1, incluido en Boot 3.2.5, sin dependencias nuevas), sucesor síncrono de `RestTemplate` con una API fluida. Al no traer una política de reintentos declarativa, esta se implementó a mano en `FootballDataClient`, con una regla explícita según el tipo de fallo: solo se reintentan ante fallos transitorios (*timeout*, error 5xx, error de red); nunca ante 429 (agotaría la cuota de 10 peticiones/minuto del plan gratuito de `football-data.org` y garantizaría otro 429), ni ante 401/403 (una clave inválida no se arregla reintentando), ni ante 404 (error determinista):

```

1  for (int intento = 1; intento <= props.maxIntentos(); intento++) {
2      try {
3          return footballRestClient.get()
4              .uri("/competitions/{codigo}/standings", codigoLiga)
5              .retrieve()
6              .body(StandingsExternoDto.class);
7      } catch (HttpClientErrorException.TooManyRequests e) {
8          throw new ApiExternaException(Motivo.RATE_LIMIT, HttpStatus.
9              TOO_MANY_REQUESTS,
10              "El proveedor externo aplico limite de peticiones (plan gratuito)");
11      } catch (HttpClientErrorException.Unauthorized | HttpClientErrorException.
12          Forbidden e) {
13          throw new ApiExternaException(Motivo.NO_AUTORIZADO, HttpStatus.
14              BAD_GATEWAY,
15              "La clave de la API externa es invalida o no fue configurada");
16      } catch (HttpClientErrorException.NotFound e) {
17          throw new ApiExternaException(Motivo.NO_ENCONTRADO, HttpStatus.
18              BAD_GATEWAY,
19              "El proveedor externo no tiene datos para esa liga");
20      } catch (HttpServerErrorException e) {
21          ultimoFallo = new ApiExternaException(Motivo.SERVIDOR, HttpStatus.
22              BAD_GATEWAY, "...");
23      } catch (ResourceAccessException e) {
24          ultimoFallo = new ApiExternaException(Motivo.TIMEOUT, HttpStatus.
25              BAD_GATEWAY, "...");
26      }
27      if (intento < props.maxIntentos()) { dormir(esperaMs); esperaMs *= 2; } //
28          backoff 400ms -> 800ms
29  }

```

Listing 6: Manejo completo de errores en el consumo de la API externa (`FootballDataClient.java`)

Sobre concurrencia: `RestTemplate` y `RestClient` bloquean el hilo que atiende la petición HTTP entrante mientras esperan al proveedor externo, mientras que `WebClient` libera ese hilo y lo reanuda cuando llega la respuesta. Para el volumen de tráfico de SGED (una escuela deportiva, no un servicio de alta concurrencia) el modelo bloqueante es aceptable siempre que el *timeout*

esté acotado (3s de conexión, 5s de lectura); un sistema con miles de peticiones concurrentes hacia proveedores externos lentos justificaría migrar a `WebClient`.

5.3 Caché de respuestas externas

Cachear una API externa no es una optimización opcional: protege contra tres riesgos concretos. Primero, evita superar el límite de peticiones del proveedor —el patrón de *rate limiting* documentado por Serbout et al. [15] exige que el cliente absorba la mayoría de las lecturas repetidas para no ser bloqueado ni penalizado por el servidor. Segundo, reduce la latencia percibida: una lectura de Redis tarda milisegundos frente a los cientos de milisegundos (o segundos, con reintentos) de una llamada de red a un tercero. Tercero, protege contra la indisponibilidad del proveedor: si `football-data.org` cae, los datos servidos desde caché siguen disponibles hasta que expira el TTL.

El TTL debe ajustarse al dominio del dato, no ser un valor arbitrario: datos meteorológicos cambian cada pocos minutos (TTL razonable: 10 min), las tasas de cambio de divisas se actualizan varias veces por hora (TTL: 1 h), y una tabla de posiciones de una liga de fútbol solo cambia cuando termina una jornada de partidos, típicamente una vez por semana —es un dato casi estático dentro de un mismo día, por lo que SGED usa un TTL de 24 h. La implementación sigue el patrón *cache-aside*: el servicio consulta primero Redis, y solo golpea la API externa en caso de *miss*, guardando el resultado con el TTL configurado:

```

1 TablaPosicionesDto enCache = leerCache(clave, meta);
2 if (enCache != null) {
3     meta.put("origen", "cache");
4     return ApiResponse.ok(enCache, "...", meta);           // HIT: no toca la
        red
5 }
6 try {
7     StandingsExternoDto externo = client.obtenerPosiciones(liga);
8     TablaPosicionesDto tabla = mapear(externo, liga);
9     guardarCache(clave, tabla, meta);                     // MISS: guarda con
        TTL 24h
10    meta.put("origen", "api");
11    return ApiResponse.ok(tabla, "...", meta);
12 } catch (ApiExternaException e) {
13    meta.put("origen", "fallback");                       // proveedor caído:
        datos de referencia, HTTP 200
14    return ApiResponse.ok(fallbackMock(liga), "...", meta);
15 }
```

Listing 7: Patron cache-aside con TTL de 24h (FutbolService.java)

Redis guarda la clave `api_externa:standings:{LIGA}` con EX 86400; si Redis está caído, el servicio no falla: registra `meta.cache = "no disponible"` y sigue respondiendo desde la API externa (evidencia: `FutbolServiceTest.posiciones_redisCaído_sigueRespondiendoDesdeLaApi`).

5.4 Tendencias: Jamstack y APIs en 2025

El patrón Jamstack (JavaScript + APIs + Markup) separa la generación de las páginas del consumo de datos: generadores de sitios estáticos como Next.js, Nuxt.js o Astro producen HTML en tiempo de compilación (*build time*) y lo hidratan o complementan con llamadas a APIs en tiempo de ejecución (*runtime*), obteniendo el rendimiento de un sitio estático con la frescura de datos dinámicos [10]. Es una evolución directa del consumo de APIs externas que documenta esta sección: si SGED expusiera sus posiciones deportivas como contenido público, un *front-end* Jamstack podría pre-renderizar la tabla en *build time* y revalidarla periódicamente contra `/api/futbol/posiciones`, delegando el mismo problema de caché a la capa de *build*.

La segunda tendencia de 2025 es la IA generativa como parte del flujo de desarrollo: herramientas como GitHub Copilot y el andamiaje (*scaffolding*) inteligente de código aceleran tareas

repetitivas, pero introducen riesgos documentados en la literatura: cuestiones de propiedad intelectual sobre el código generado y la responsabilidad última del desarrollador sobre lo que integra sin revisar [11]. SGED no es ajeno a esta tendencia ni a sus riesgos: la sección 13 declara explícitamente el alcance del uso de IA en esta práctica, precisamente para que la evaluación no dependa de asumir qué se generó y qué se verificó.

6 Seguridad: auditoría OWASP (Bloque 3)

La seguridad se verificó con el *script* `scripts/audit-owasp.sh`, que ejecuta peticiones `curl` reales contra el sistema levantado, guarda la evidencia cruda en `docs/mediciones/sec/` con fecha y *commit*, y comprueba cada control en ambas direcciones: el comportamiento seguro esperado *y* que no se haya roto el uso legítimo. Se cubren los diez riesgos del OWASP Top 10:2021 [13] en su alcance aplicable, más la categoría XSS (Cross-Site Scripting) de forma explícita.

6.1 A01 — Broken Access Control

Se comprueba con un usuario autenticado sin rol administrativo que las operaciones administrativas devuelvan 403, y que las lecturas permitidas al rol `USER` sigan funcionando:

Petición	Resultado
POST <code>/api/estudiantes/operaciones/desactivar-categoria</code>	403
GET <code>/api/personas</code>	403
GET <code>/api/personas/cedula/0000000000</code>	403
POST <code>/api/categorias</code>	403
DELETE <code>/api/categorias/1</code>	403
PUT <code>/api/personas/1</code>	403
GET <code>/api/usuarios</code>	403
GET <code>/api/categorias/activas</code> (uso legítimo)	200
GET <code>/api/estados_generales</code> (uso legítimo)	200

El error se devuelve como `application/problem+json` conforme al RFC 9457 [12], con `title: Forbidden` y el detalle «No tiene permisos para acceder a este recurso». La auditoría limpia al inicio su propio contador de intentos de login en Redis, porque de lo contrario el propio control A07 bloquearía por IP la ejecución y A01 devolvería 401 en todo, un resultado que parece correcto y no lo es (ver sección 9).

6.2 A02 — Cryptographic Failures

La evidencia `docs/mediciones/sec/a02-tls.txt` muestra el apretón de manos TLS contra el frontend: **TLS 1.3**, cifrado `TLS_AES_256_GCM_SHA384`, intercambio de claves `X25519` y firma `RSASSA-PSS`. El JWT se transporta en *cookie* `HttpOnly + Secure + SameSite` y las contraseñas se almacenan con un *hash* seguro [9].

6.3 A03 — Injection

Se comprueba que el sistema rechaza con 422 las peticiones que no cumplen la validación de Bean Validation, en lugar de ejecutarlas. La evidencia A03 muestra la validación de `POST /api/estudiantes` con todos los campos obligatorios ausentes, devolviendo **errores** por campo en `application/problem+json`. Las consultas se ejecutan mediante Spring Data JPA parametrizado y los procedimientos almacenados se invocan con `@Procedure`; no se concatena SQL.

6.4 A04 — Insecure Design

El diseño inseguro se previene en la capa de diseño, no solo con *fixes* puntuales. La evidencia A04 comprueba tres reglas de negocio:

- **Política de contraseñas** aplicada en el servidor: POST /api/auth/registro con contraseña corta devuelve 422.
- **Catálogos obligatorios**: POST /api/estudiantes con una categoría inexistente devuelve 404 (la regla de negocio se valida contra el catálogo real, no solo el tipo).
- **Credenciales duplicadas** rechazadas: el registro con un `username` ya usado devuelve 409.

El modelo de amenazas y las decisiones de diseño se documentan en la sección 9.

6.5 A05 — Security Misconfiguration

Toda respuesta del backend incluye las cabeceras de seguridad:

```
1 X-Content-Type-Options: nosniff
2 X-Frame-Options: DENY
3 Content-Security-Policy: default-src 'self'; frame-ancestors 'none'
4 Cache-Control: no-cache, no-store, max-age=0, must-revalidate
```

Listing 8: Cabeceras de seguridad (evidencia A05)

6.6 A06 — Vulnerable and Outdated Components

El inventario de dependencias se verifica con sus versiones y se documenta en `docs/mediciones/sec/a06-componentes`. Spring Boot 3.2.5, jjwt 0.12.5, springdoc-openapi 2.3.0, Flyway 10.12.0, Angular 21.2.x y TypeScript 5.9.2. Las imágenes de PostgreSQL y Redis están fijadas por digest SHA-256 en `docker-compose.yml`, de modo que dos construcciones del mismo *commit* usan exactamente los mismos binarios.

6.7 A07 — Identification and Authentication Failures

La evidencia A07 comprueba el bloqueo por fuerza bruta: los intentos 1–5 de login con credenciales incorrectas devuelven 401 y el sexto devuelve 429 Too Many Requests, dejando la IP bloqueada durante 15 minutos. El controlador de intentos se lleva en Redis por IP de origen (`login_attempts:<ip>`), no por usuario.

6.8 A09 — Security Logging and Monitoring Failures

El registro de auditoría escribe eventos de autenticación con marca de tiempo, IP de origen y sujeto (evidencia A09): AUTH_LOGIN_OK en nivel INFO y AUTH_LOGIN_FAIL en nivel WARN.

6.9 XSS — Cross-Site Scripting

La protección ante XSS se verifica por tres vías (evidencia `xss.txt`): (1) la cabecera `Content-Security-Policy: default-src 'self'; frame-ancestors 'none'` está presente; (2) un payload `<script>` enviado en POST /api/categorias se *almacena* devuelto con 201, pero el servidor lo sirve siempre como `application/json`, que el navegador no interpreta como HTML; y (3) Angular lo escapa por defecto en el SPA al renderizarlo. La evidencia comprueba que el *payload* viaja como dato escapable, no como *HTML* ejecutable.

6.10 Resultado de la auditoría

Control	Comportamiento verificado	Estado
A01 Control de acceso	403 en recursos administrativos	OK
A02 Criptografía	TLS 1.3, cookie HttpOnly	OK
A03 Inyección	Validación server-side, JPA parametrizado	OK
A04 Diseño inseguro	Reglas de negocio: 422/404/409	OK
A05 Configuración	Cabeceras de seguridad presentes	OK
A06 Componentes	Inventario con versiones, digests fijos	OK
A07 Autenticación	6. ^o intento → 429 (15 min)	OK
A09 Registro	AUTH_LOGIN_OK/AUTH_LOGIN_FAIL	OK
XSS	CSP + application/json + escape Angular	OK

7 Migraciones Flyway y reproducibilidad (Bloque 4)

Las migraciones versionadas V1–V7 se alinearon con las entidades JPA y con el esquema consolidado `db/schema.sql`:

- V1: esquema `academico` + tablas de personas y usuarios.
- V2: catálogos y `academico.estudiantes`.
- V3: entrenadores con `id_usuario`, experiencia y certificación, con FK a `academico`.
- V4: claves foráneas a `academico`.
- V5: funciones `academico` con `id_categoria` INT.
- V6: procedimientos almacenados.
- V7: *seed* idempotente (`admin/Admin2026!`).

La verificación se hizo con bases de datos limpias aplicando V1–V7 con Flyway y arrancando la aplicación con `ddl-auto=validate`, confirmando que el esquema migrado coincide con las entidades. El arranque completo del sistema es reproducible con `make up` [14].

8 Pruebas de carga (Bloque 5.3)

[Pendiente de Integrante B. Sección reservada.]

Esta sección documenta las pruebas de carga sobre el *endpoint* principal de la API con k6 [7]: 3 corridas independientes de 50 usuarios virtuales durante 30 segundos, con análisis de percentiles e intervalo de confianza al 95 %.

Corrida	P50	P95	P99	req/s	% error
Corrida 1	[_____]	[_____]	[_____]	[_____]	[_____]
Corrida 2	[_____]	[_____]	[_____]	[_____]	[_____]
Corrida 3	[_____]	[_____]	[_____]	[_____]	[_____]
Media	[_____]	[_____]	[_____]	[_____]	[_____]
IC 95 %	[_____]	[_____]	[_____]	[_____]	[_____]

Umbral objetivo: P95 menor a 200 ms con caché caliente y menor a 500 ms con caché frío (ISO/IEC 25010 [8]).

[Pendiente de Integrante B: completar la tabla con los valores de las corridas k6-run1..3.json, el análisis del *script* `scripts/perf-analysis.py` y la conclusión frente al umbral.]

9 Reflexión y modelo de amenazas (Bloque 5.4)

9.1 Modelo de amenazas y decisiones de diseño

El análisis de diseño inseguro (A04) no es una lista de *fixes*, sino un modelo de amenazas previo a la implementación. Para SGED, cuyo dato más sensible es la información personal de menores de edad (cédula, fecha de nacimiento, asistencia y evaluaciones), se decidió:

- **Validación de negocio en el servidor**, no solo en el *frontend*: el cliente Angular valida para dar retroalimentación inmediata, pero el backend rechaza cualquier violación con 422/404/409 (evidencia A04).
- **Autenticación basada en *cookie* HttpOnly** en lugar de JWT en `localStorage`: el token no es legible por *scripts*, reduciendo la superficie de exfiltración XSS.
- **Política de contraseñas obligatoria** y bloqueo por fuerza bruta con Redis: el límite se aplica por IP para impedir el barrido de credenciales sin penalizar a un usuario legítimo.
- **Content-Type siempre `application/json` + CSP `default-src 'self'`**: un *payload* almacenado no puede convertirse en HTML ejecutable; el escape de Angular es la segunda línea de defensa.

9.2 Lección transversal: una medición que no se ejecuta contra el sistema real no es evidencia

Durante la práctica se materializó tres veces el mismo error de medición: *un indicador daba verde sobre un sistema que no hacía lo que el indicador afirmaba*. La primera vez, en la Tercera Entrega, la cobertura de JaCoCo se medía sobre `target/` con clases previas a la reestructuración e incluía paquetes inexistentes. La segunda, la auditoría A01 devolvía 401 en todo porque el propio control A07 había bloqueado la IP de la máquina de auditoría: ese resultado parece correcto y no lo es, porque probaba autenticación, no autorización. La tercera, el registro de usuarios estaba roto contra la base de datos mientras sus pruebas unitarias pasaban, porque usan dobles de prueba y la restricción la aplica PostgreSQL.

La corrección estructural fue la misma en los tres casos: la medición debe ejecutarse contra el sistema real, desde estado limpio, y comprobar las dos direcciones (el rechazo seguro *y* el uso legítimo). La auditoría de esta práctica incorpora ambas propiedades: limpia su propio contador de intentos al inicio y verifica por cada control que el recurso legítimo sigue devolviendo 200. Esa es la diferencia entre una evidencia que acompaña a un resultado y una evidencia que lo produce.

9.3 Contribución de la práctica

El aporte de la práctica no fue solo añadir documentación y controles, sino *verificar con evidencia reproducible* lo que el sistema ya afirmaba tener, y corregir en su origen los desajustes encontrados: las migraciones Flyway no estaban alineadas con el esquema y hoy lo están; el inventario de componentes A06 carecía de versiones y hoy las declara; y la auditoría OWASP ampliada cubre A04, A06, A09 y XSS con la misma disciplina que los controles originales.

10 Autoría y honestidad académica

La autoría es verificable en el historial de `git` de la rama `feature/u4-docs-seguridad` (`git log -author`). Los bloques de la práctica se asignaron así:

Integrante	Bloques
Velez Lopez Ricardo Elias	Bloques 1–4, auditoría OWASP (sección 6), informe
Integrante A	Bloque 5.1: API externa (sección 4)
Integrante B	Bloque 5.3: pruebas de carga (sección 8)

[Pendiente de completar nombres, correos institucionales y enlaces a los *commits* de los integrantes A y B.]

11 Conclusiones

1. La API REST de SGED quedó documentada y consultable en `/api/docs` (Swagger UI) y `/api/api-docs` (OpenAPI 3.0), con anotaciones `@Tag/@Operation/@Schema` en controladores y DTOs. La documentación es consumible por herramientas, no un PDF suelto sin fuente verificable (Bloques 1–2).
2. La auditoría de seguridad comprueba 9 controles (A01–A07, A09 y XSS) con peticiones `curl` reproducibles guardadas como evidencia cruda. Todos los comportamientos verificados son los esperados, y la auditoría comprueba la doble dirección: el rechazo seguro y el uso legítimo intacto.
3. El sistema incorpora defensas de diseño, no solo de código: bloqueo por fuerza bruta en Redis, validación de negocio en el servidor, cookie `HttpOnly` y CSP con `default-src 'self'`. La protección ante XSS se evidencia por tres vías independientes.
4. Las migraciones Flyway V1–V7 quedaron alineadas con las entidades y el esquema consolidado, verificadas contra bases de datos limpias con `ddl-auto=validate`, y el arranque completo es reproducible con un solo comando (`make up`).
5. El uso de herramientas de inteligencia artificial en el desarrollo se declara de forma explícita y transparente en la sección 13; no sustituye la comprensión ni la autoría, y se registra con la misma disciplina que el resto de la práctica.

12 Trabajo futuro

- Completar los bloques 5.1 (API externa) y 5.3 (pruebas de carga) con los resultados de los integrantes A y B, y recompilar este informe.
- Incorporar el escaneo OWASP Dependency-Check al CI cuando la base NVD sea accesible en el entorno de integración.
- Reemplazar el certificado TLS autofirmado por uno de confianza antes de cualquier despliegue no académico.
- Asignar el DOI de Zenodo a la etiqueta `v1.0.0-rc` y publicar el video de demostración.
- Extender la cobertura de pruebas a los módulos nuevos del dominio deportivo expuestos en la Entrega Final.

13 Declaración de uso de inteligencia artificial

De acuerdo con la guía de transparencia del SWEBOK v4.0 [16] y con la política de honestidad académica de la asignatura, se declara explícitamente que en el desarrollo de esta práctica se utilizaron herramientas de asistencia por inteligencia artificial. El alcance y los límites de ese uso se precisan para que la evaluación sea justa:

- **Lo que la IA asistió:** redacción de la estructura de este informe en $\text{\LaTeX}$, generación de esqueletos de *scripts* de auditoría, verificación de la sintaxis de las migraciones Flyway y ayuda en la depuración de mensajes de error de la cadena de herramientas (Maven, Flyway, Docker Compose).
- **Lo que no se delegó a la IA:** el diseño del sistema, el modelado de amenazas, la escritura de la lógica de negocio, la interpretación de los resultados de las mediciones, las decisiones de arquitectura y la redacción final de las conclusiones. Cada resultado se verificó ejecutando el comando correspondiente contra el sistema real.
- **Traza:** los *commits* de la rama `feature/u4-docs-seguridad` permiten al docente auditar el trabajo de cada integrante; la evidencia cruda en `docs/mediciones/` permite reproducir cada cifra citada.

A Anexo: Taxonomía de Bloom aplicada a la práctica

Siguiendo la taxonomía revisada de Anderson y Krathwohl [1] (original de Bloom [3]), la práctica se ubica principalmente en los niveles *crear* y *evaluar*: los bloques no solo implementan una API documentada, sino que construyen y verifican evidencia para juzgar si el sistema cumple los umbrales exigidos. La tabla relaciona cada nivel con las actividades concretas de la práctica.

Nivel	Actividad en la práctica	Artefacto
Recordar	Conceptos REST, OWASP, TLS, caché	Secciones 1–3
Comprender	Flujo MVC Spring Boot y JWT en cookie	<code>docs/diagramas/</code>
Aplicar	Configurar Swagger, anotar DTOs, escribir migraciones	Bloques 1–4
Analizar	Auditar controles OWASP y descomponer fallos de medición	<code>docs/mediciones/sec/</code>
Evaluar	Juzgar cada control frente a umbrales exigidos	Tabla de la sección 6
Crear	Construir la evidencia reproducible y este informe	<code>Makefile</code> , <code>main.tex</code>

Referencias

- [1] Lorin W. Anderson and David R. Krathwohl. *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. Longman, 2001.
- [2] Enes Bajrami, Agon Memeti, Florim Idrizi, and Ermira Memeti. Comparative analysis of SOAP and REST APIs: Systematic review and performance evaluation with Python. *Journal of Natural Sciences and Mathematics of UT (JNSM)*, 9(17–18):228–243, 2024.
- [3] Benjamin S. Bloom, editor. *Taxonomy of Educational Objectives: The Classification of Educational Goals — Handbook I: Cognitive Domain*. David McKay Company, 1956.
- [4] Anil Kumar Chitiprolu. A framework for automated transformation of legacy SOAP-based services to RESTful APIs. *International Journal of Engineering, Science and Information Technology (IJEITY)*, 6(3), 2026.
- [5] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [6] Football Data Co. Football-data.org — API de datos de fútbol. <https://www.football-data.org/>, 2026.
- [7] Grafana Labs. k6 — load testing tool for engineering teams. <https://k6.io/docs/>, 2026.
- [8] ISO/IEC. ISO/IEC 25010:2011 — systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — system and software quality models, 2011.
- [9] M. B. Jones, J. Bradley, and N. Sakimura. JSON web token (JWT). Request for Comments 7519, Internet Engineering Task Force (IETF), May 2015. <https://www.rfc-editor.org/rfc/rfc7519>.
- [10] Dragana Markovic, Milic Scekcic, Alessio Bucaioni, and Antonio Cicchetti. Could jamstack be the future of web applications architecture? In *Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing (SAC '22)*. ACM, April 2022.
- [11] Arghavan Moradi Dakhel, Vahid Majdinasab, Amin Nikanjam, Foutse Khomh, Michel C. Desmarais, and Zhen Ming Jiang. GitHub Copilot AI pair programmer: Asset or liability? *Journal of Systems and Software*, 203:111734, September 2023.
- [12] M. Nottingham, E. Wilde, and S. Dalal. Problem details for HTTP APIs. Request for Comments 9457, Internet Engineering Task Force (IETF), July 2023. <https://www.rfc-editor.org/rfc/rfc9457>.
- [13] OWASP Foundation. OWASP top 10:2021 — the ten most critical web application security risks. <https://owasp.org/Top10/>, 2021.
- [14] Redgate. Flyway documentation — migrations. <https://documentation.red-gate.com/flyway/>, 2026.
- [15] Souhaila Serbout, Amine El Malki, Cesare Pautasso, and Uwe Zdun. API rate limit adoption — a pattern collection. In *Proceedings of the 28th European Conference on Pattern Languages of Programs (EuroPLoP 2023)*. ACM, July 2023.
- [16] Hironori Washizaki, editor. *SWEBOK Guide v4.0: Guide to the Software Engineering Body of Knowledge*. IEEE Computer Society, 2024.